

CS 182 Lecture 2: Optimization

Deep Neural Networks

November 17, 2025

1 Implicit Regularization of Gradient Descent

1.1 Recap: Geometry and Convergence of Gradient Descent

From the previous lecture, we analyzed the convergence of Gradient Descent (GD) for linear regression (least squares). A key takeaway is that the geometric properties of the loss function are critical for determining practical hyperparameters, such as the learning rate.

For the least squares problem, the convergence of GD is guaranteed if the learning rate η satisfies:

$$\eta < \frac{1}{\lambda_{\max}(\mathbf{X}^T \mathbf{X})} = \frac{1}{\sigma_{\max}^2(\mathbf{X})} \quad (1)$$

where $\lambda_{\max}$ is the largest eigenvalue and $\sigma_{\max}$ is the largest singular value. The rate of convergence is determined by the condition number of $\mathbf{X}^T \mathbf{X}$, which is the ratio $\frac{\lambda_{\max}}{\lambda_{\min}} = \left(\frac{\sigma_{\max}}{\sigma_{\min}}\right)^2$.

This concept generalizes beyond quadratic loss functions. Properties like **L-smoothness** and **μ -strong convexity** provide upper and lower quadratic bounds on general loss functions, allowing us to extend these geometric intuitions. The convergence rate in these more general cases depends on the ratio L/μ .

Mention of this was exclusive to the lecture recording, which I would've missed if I only looked at the slides.

1.2 Ridge Regression in the SVD Basis

To understand regularization, we analyze Ridge Regression. The solution to the Ridge objective is given by:

$$\vec{w}_* = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \vec{y} \quad (2)$$

An equivalent form, known as the kernel form, is $\vec{w}_* = \mathbf{X}^T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})^{-1} \vec{y}$.

Let's analyze the solution in the basis defined by the Singular Value Decomposition (SVD) of the data matrix $\mathbf{X}$. Let $\mathbf{X} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$. Substituting this into the solution for $\vec{w}_*$:

$$\begin{aligned} \vec{w}_* &= ((\mathbf{U} \mathbf{\Sigma} \mathbf{V}^T)^T (\mathbf{U} \mathbf{\Sigma} \mathbf{V}^T) + \lambda \mathbf{I})^{-1} (\mathbf{U} \mathbf{\Sigma} \mathbf{V}^T)^T \vec{y} \\ &= (\mathbf{V} \mathbf{\Sigma}^T \mathbf{U}^T \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T + \lambda \mathbf{V} \mathbf{V}^T)^{-1} \mathbf{V} \mathbf{\Sigma}^T \mathbf{U}^T \vec{y} \quad (\text{since } \mathbf{I} = \mathbf{V} \mathbf{V}^T) \\ &= (\mathbf{V} (\mathbf{\Sigma}^T \mathbf{\Sigma} + \lambda \mathbf{I}) \mathbf{V}^T)^{-1} \mathbf{V} \mathbf{\Sigma}^T \mathbf{U}^T \vec{y} \quad (\text{since } \mathbf{U}^T \mathbf{U} = \mathbf{I}) \\ &= \mathbf{V} (\mathbf{\Sigma}^T \mathbf{\Sigma} + \lambda \mathbf{I})^{-1} \mathbf{V}^T \mathbf{\Sigma}^T \mathbf{U}^T \vec{y} \\ &= \mathbf{V} (\mathbf{\Sigma}^T \mathbf{\Sigma} + \lambda \mathbf{I})^{-1} \mathbf{\Sigma}^T \mathbf{U}^T \vec{y} \end{aligned}$$

The matrix $(\mathbf{\Sigma}^T \mathbf{\Sigma} + \lambda \mathbf{I})$ is a diagonal matrix with diagonal entries $\sigma_i^2 + \lambda$. Its inverse is also diagonal with entries $1/(\sigma_i^2 + \lambda)$. This allows us to express the solution as a sum over the singular components:

$$\vec{w}_* = \sum_{i=1}^{\text{rank}(\mathbf{X})} \vec{v}_i \left(\frac{\sigma_i}{\sigma_i^2 + \lambda} \right) (\vec{u}_i^T \vec{y}) \quad (3)$$

Here, $\vec{v}_i$ and $\vec{u}_i$ are the right and left singular vectors, respectively. The term $\vec{u}_i^T \vec{y}$ is a scalar projection.

The behavior of the solution is governed by the scaling factor $\frac{\sigma_i}{\sigma_i^2 + \lambda}$:

- If $\lambda \gg \sigma_i^2$ (i.e., for directions with small singular values), the scaling factor approaches 0. The components of the solution in these directions are suppressed.
- If $\sigma_i^2 \gg \lambda$ (i.e., for directions with large singular values), the scaling factor is approximately $\frac{\sigma_i}{\sigma_i^2} = \frac{1}{\sigma_i}$.

I like the format of this better than the slides, as it explains the 2 outcomes very clearly.

Small singular values correspond to directions of low variance in the data. Ridge regularization, therefore, prevents the model from fitting patterns in these low-variance directions, which are often dominated by noise. This is how Ridge helps prevent overfitting.

1.3 Gradient Descent and Early Stopping

The standard Gradient Descent update for least squares is:

$$\vec{w}_{t+1} = \vec{w}_t - 2\eta \mathbf{X}^T (\mathbf{X} \vec{w}_t - \vec{y}) \quad (4)$$

An important observation is that the update step $\Delta \vec{w}_t$ is always in the column space of $\mathbf{X}^T$, which is the row space of $\mathbf{X}$ (the span of the data). If initialized at $\vec{w}_0 = \vec{0}$, the solution $\vec{w}_t$ will always remain in the span of the data.

For comparison, the GD update for the Ridge objective includes a term often called **weight decay**:

$$\vec{w}_{t+1} = (1 - 2\eta\lambda) \vec{w}_t - 2\eta \mathbf{X}^T (\mathbf{X} \vec{w}_t - \vec{y}) \quad (5)$$

The term $(1 - 2\eta\lambda)$ shrinks the weights at each step.

Now, let's analyze the standard GD update in the SVD basis. Let $\tilde{\vec{w}}_t = \mathbf{V}^T \vec{w}_t$ be the weights in the new basis.

$$\begin{aligned} \mathbf{V}^T \vec{w}_{t+1} &= \mathbf{V}^T \vec{w}_t - 2\eta \mathbf{V}^T (\mathbf{V} \Sigma^T \mathbf{U}^T) (\mathbf{U} \Sigma \mathbf{V}^T \vec{w}_t - \vec{y}) \\ \tilde{\vec{w}}_{t+1} &= \tilde{\vec{w}}_t - 2\eta \Sigma^T \mathbf{U}^T (\mathbf{U} \Sigma \tilde{\vec{w}}_t - \vec{y}) \\ \tilde{\vec{w}}_{t+1} &= \tilde{\vec{w}}_t - 2\eta \Sigma^T (\Sigma \tilde{\vec{w}}_t - \mathbf{U}^T \vec{y}) \end{aligned}$$

Since $\Sigma^T \Sigma$ is diagonal, the system decouples. For the i -th component:

$$\tilde{w}_{t+1,i} = \tilde{w}_{t,i} - 2\eta \sigma_i^2 \tilde{w}_{t,i} + 2\eta \sigma_i (\vec{u}_i^T \vec{y}) = (1 - 2\eta \sigma_i^2) \tilde{w}_{t,i} + 2\eta \sigma_i \tilde{y}_i \quad (6)$$

The effective step size in the direction of the i -th singular vector is proportional to σ_i^2 . This means:

- GD takes large steps in directions corresponding to large singular values.
- GD takes small steps in directions corresponding to small singular values.

This leads to the concept of **Early Stopping**. Gradient Descent will first converge rapidly along the high-variance directions. If we stop the algorithm before it has fully converged, it will not have had enough time to move significantly in the low-variance directions. This effectively prevents the model from fitting the noise in the data, achieving a similar regularizing effect as Ridge regression. This phenomenon is known as the **implicit regularization** of gradient descent.

2 Stochastic Gradient Descent (SGD)

2.1 Motivation and Algorithm

While GD is effective for convex problems, training deep neural networks involves non-convex loss landscapes. Furthermore, GD is computationally expensive for large datasets, as it requires computing the gradient over the entire dataset for a single update.

Stochastic Gradient Descent (SGD) addresses these issues. It assumes the loss function can be decomposed into a sum over individual data points:

$$f(\theta) = \frac{1}{n} \sum_{i=1}^n f_i(\theta) \quad (7)$$

Instead of computing the full gradient $\nabla f(\theta)$, SGD computes the gradient on a small, randomly sampled subset of the data called a **mini-batch**, $\mathcal{B}$. The update rule is:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla f_i(\theta_t) \quad (8)$$

The gradient estimate is noisy but unbiased, assuming the mini-batch is sampled uniformly at random:

$$\mathbb{E}_{i \sim \text{Uniform}}[\nabla f_i(\theta)] = \nabla f(\theta) \quad (9)$$

2.2 Properties of SGD

- **Computationally Efficient:** Each step is much faster than a full GD step.
- **Noise helps escape local minima:** The stochasticity in the gradient estimate can help the optimization process escape sharp local minima or saddle points and find better, flatter minima, which often leads to better generalization.
- **Convergence:** Traditional optimization theory suggests that for SGD to converge, the learning rate η must decrease over time (a learning rate schedule). However, in the context of deep learning (often over-parameterized models), SGD can converge even with a constant step size.

There is an image from the slides that is missing from these notes; this is a limitation of this method.

3 Momentum

3.1 Motivation: Oscillations in Gradient Descent

The learning rate η for GD is limited by the largest singular value, $\sigma_{\max}$. This choice of η can lead to very slow convergence in directions with smaller singular values. If we try to increase η to speed up convergence, the update term for the largest singular values, $(1 - 2\eta\sigma_{\max}^2)$, can become negative, causing oscillations in the optimization path. The goal is to accelerate convergence while damping these oscillations.

3.2 The Momentum Update

A common technique to smooth oscillations is averaging. Instead of a simple moving average, which is memory-intensive, we can use an exponentially decaying average of past gradients. This is the idea behind momentum. It introduces a "velocity" or "state" variable, $\tilde{z}$, that accumulates a history of gradients.

The momentum update consists of two steps:

$$\vec{z}_{k+1} = \beta \vec{z}_k + \nabla f(\vec{w}_k) \quad (\text{A common variant of the update}) \quad (10)$$

$$\vec{w}_{k+1} = \vec{w}_k - \eta \vec{z}_{k+1} \quad (11)$$

Alternatively, using the exponential moving average form:

$$\vec{z}_{k+1} = \beta \vec{z}_k + (1 - \beta) \nabla f(\vec{w}_k), \quad \text{with } \vec{z}_0 = \vec{0} \quad (12)$$

$$\vec{w}_{k+1} = \vec{w}_k - \eta \vec{z}_{k+1} \quad (13)$$

Here, $\beta \in [0, 1)$ is the momentum parameter. If we expand the recurrence for $\vec{z}_k$, we see it is an exponentially weighted average of past gradients:

$$\vec{z}_k = (1 - \beta) \sum_{i=0}^{k-1} \beta^{k-1-i} \nabla f(\vec{w}_i) \quad (14)$$

The most recent gradients receive the highest weight.

Intuition: Momentum acts like a ball rolling down a hill. It builds up speed in directions of consistent gradient and smooths out directions where the gradient oscillates. This has two main benefits:

This is something I really like to see in notes, as it provides intuitive understanding, not just mathematical justifications.

1. It smooths the optimization trajectory, especially for SGD where gradients are noisy.
2. It can provably accelerate convergence. For convex quadratic problems, GD's convergence rate depends on the condition number κ , while momentum's rate depends on $\sqrt{\kappa}$, which is a significant improvement.

The main drawback is the introduction of a new hyperparameter, β , that needs to be tuned.

4 Analysis of SGD Convergence

We will now set up a proof to show that SGD can converge with a constant step size in a specific setting.

4.1 Problem Setup

Consider an underdetermined linear system $\mathbf{X}\vec{w} = \vec{y}$, where $\mathbf{X} \in \mathbb{R}^{n \times d}$ with $d > n$ (a "wide" matrix) and $\mathbf{X}$ has full row rank. There are infinitely many solutions. Let $\vec{w}_*$ be the minimum-norm solution. We will analyze SGD with batch size 1, initialized at $\vec{w}_0 = \vec{0}$.

We transform the problem by considering the error vector $\vec{q}_t = \vec{w}_t - \vec{w}_*$. The system becomes $\mathbf{X}\vec{q} = \vec{0}$. The loss function for the original problem is $L(\vec{w}) = \frac{1}{2} \|\mathbf{X}\vec{w} - \vec{y}\|^2$. In the error space, we are trying to solve $\mathbf{X}\vec{q} = \vec{0}$.

Using the SVD of $\mathbf{X}$, we can show that the dynamics of SGD are confined to the row space of $\mathbf{X}$. We can therefore reduce the problem from d dimensions to n dimensions, analyzing an equivalent square system $\tilde{\mathbf{X}}\vec{z} = \vec{0}$, where $\vec{z} \in \mathbb{R}^n$.

The loss can be expressed as $L(\vec{z}) = \|\tilde{\mathbf{X}}\vec{z}\|^2 = \vec{z}^T \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \vec{z}$. The SGD update for batch size 1 (sampling the I_t -th data point) in this reduced space is:

$$\vec{z}_{t+1} = \vec{z}_t - 2\eta \tilde{x}_{I_t} (\tilde{x}_{I_t}^T \vec{z}_t) \quad (15)$$

where $\tilde{x}_{I_t}$ is the I_t -th row of $\tilde{\mathbf{X}}$.

I like this problem setup section, as it gives very clear definitions (the same ones in the slides) with more elaborate explanations.

4.2 Proof Strategy via a Lyapunov Function

To prove that the loss $L(\vec{z}_t)$ converges to 0, we can use a technique analogous to a Lyapunov function in control theory. We want to show that the expected loss decreases at every step. Specifically, we want to establish a contraction:

$$\mathbb{E}[L(\vec{z}_{t+1})|\vec{z}_t] \leq (1 - C)L(\vec{z}_t) \quad (16)$$

for some constant $C > 0$. If this holds, the expected loss follows a geometric progression and must converge to zero.

To analyze $\mathbb{E}[L(\vec{z}_{t+1})]$, we can write:

$$\begin{aligned} L(\vec{z}_{t+1}) &= \left\| \tilde{\mathbf{X}}(\vec{z}_t + (\vec{z}_{t+1} - \vec{z}_t)) \right\|^2 \\ &= \left\| \tilde{\mathbf{X}}\vec{z}_t \right\|^2 + 2(\tilde{\mathbf{X}}\vec{z}_t)^T \tilde{\mathbf{X}}(\vec{z}_{t+1} - \vec{z}_t) + \left\| \tilde{\mathbf{X}}(\vec{z}_{t+1} - \vec{z}_t) \right\|^2 \\ &= L(\vec{z}_t) + \text{Term A} + \text{Term B} \end{aligned}$$

The proof, to be continued, would involve taking the expectation of Term A and Term B and showing that their sum is negative and proportional to $L(\vec{z}_t)$, leading to the desired contraction.

Interestingly, it stops the proof here; this is because the lecture video itself stops and continues it next lecture.